Project Executable Files

Date	02 July 2026
Team ID	CCAP-01
Project Name	Credit Card Approval Prediction
Maximum Marks	3 Marks

Project Executable Files

This section contains all executable and deployable files for the Credit Card Approval Prediction project. All files are organized as described below and are ready to be uploaded to the submission platform so the evaluator can run or deploy the solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	NA – SQLite tables are created automatically at startup by init_history_db() in app.py
5	Environment configuration file (.env.example or similar)	Yes
6	Deployed application URL (if hosted)	Yes
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA – web application
8	Dockerfile / Containerization config (if applicable)	Yes
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure

Overview of the project's file and folder structure, based on the routes and imports in app.py:

```
Credit-Card-Approval-Prediction/
  app.py                  - Main Flask application (routes, auth, history)
  predict.py              - CreditApprovalPredictor (single & batch prediction)
  preprocessing.py        - Feature engineering, TARGET_COLUMN, build_single_record()
  train.py                - Trains and compares ML models, saves the best pipeline
  requirements.txt        - Python dependencies
  .env.example            - Sample environment configuration (SECRET_KEY, Google OAuth)
  .gitignore              - Excludes .venv, .env, instance/, uploads/, history files
  templates/              - Jinja2 HTML templates
    index.html, prediction.html, login.html, signup.html,
    about.html, features.html, model.html, admin.html,
    contact.html, batch_results.html, 404.html, 500.html
  static/                 - CSS and JavaScript assets
  models/                 - Saved model pipeline (model_metadata.pkl)
  reports/                - model_comparison.csv and reports/plots/
  uploads/                - Runtime CSV uploads & batch prediction outputs (gitignored)
  prediction_history.db   - SQLite database: prediction_history & users tables (runtime)
  .github/workflows/ci.yml - CI pipeline (installs requirements, runs pytest)
```

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	Localhost (127.0.0.1:5000)
Login Credentials (Demo)	Username: anu@gmail.com | Password: anu123
Platform / Hosting Provider	Local Flask Server (Python Flask)
Repository Link	https://github.com/penumakadharshini07-del/Credit-Card-Approval-Prediction
Demo Video Link	https://drive.google.com/file/d/1kIUm-oP8u1lnr80BFR3EVItkxLrdccuK/view?usp=sharing

Step 4: Run Instructions

Step-by-step instructions for the evaluator to run the project locally:

```
Pre-requisites:
Python 3.11+, pip, and (optionally) a Google Cloud OAuth client for Google Sign-In.

Step 1: Clone the repository
  git clone <repository-url>
  cd Credit-Card-Approval-Prediction

Step 2: Create a virtual environment and install dependencies
  python -m venv .venv
  .venv\Scripts\activate        (Windows)  |  source .venv/bin/activate  (macOS/Linux)
  pip install -r requirements.txt

Step 3: Configure environment variables
  copy .env.example to .env
  set SECRET_KEY, and GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET / GOOGLE_REDIRECT_URI
  if Google Sign-In is required

Step 4: Train the model (creates models/model_metadata.pkl and reports/model_comparison.csv)
  python train.py

Step 5: Start the application
  python app.py

Step 6: Open the browser at
  http://127.0.0.1:5000
```

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Google Sign-In requires GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET / GOOGLE_REDIRECT_URI to be set in .env; without them, only local email/phone login works.	Configure Google OAuth credentials, or use local signup/login instead.
2	The prediction model must be trained first (python train.py) before /predict or /batch will work.	Run train.py once after setup; app.py shows a warning if the model is missing.